

This page is left blank intentionally

Q1.

(a) Explain the **Merge Sort** algorithm in your own words. Explain its complexity when the size of the input is n .

(5 marks)

(b) You have designed an **Insertion Sort** algorithm for sorting n integers in ascending order. Give an example of an input for which the **Insertion Sort** algorithm runs in $O(n)$ time. What is the complexity of **Merge Sort** and **Heap Sort** algorithms on the same input?

(5 marks)

Q2.

(a) You are given two arrays A and B of integers sorted in ascending order. All integers in these two arrays are unique. The length of the array A is n , and the length of the array B is m . Design an algorithm for finding the k -th smallest element in $A \cup B$ (all the integers in both A and B), when $k < m + n$. Your algorithm should be asymptotically faster than $O(m + n)$.

(5 marks)

(b) You are given a sorted array A of n integers in ascending order, and another integer q . Design an algorithm that will either find two integers r and s in A such that $r + s = q$, or report that no such pair r, s exists. Your algorithm should run on $O(n)$ time.

(5 marks)

Q3.

(a) Arrange the following asymptotic complexities in ascending order:
 $O(2^{0.3n})$, $O(n^{0.5})$, $O(\log^2 n)$, $O(n^2 \log n)$, $O(n)$, $O(n \log n)$.

(5 marks)

(b) We add a new operation **multiPop(i)** to the stack ADT that pops i items from a stack. Explain clearly why the amortized complexity of the **multiPop(i)** operation is $O(1)$.

(5 marks)

Q4.

(a) Explain clearly the **inorder**, **preorder** and **postorder** traversals of a tree. What are the complexities of these three traversal algorithms?

(5 marks)

(b) Explain the **heap** data structure clearly. What is the complexity of maintaining the **heap** property when the element at the root of the heap is deleted? Explain how a priority queue can be implemented as a **heap**.

(5 marks)

Q5.

(a) Explain clearly Kruskal's **minimum spanning tree** algorithm. Explain briefly how it is implemented.

(5 marks)

(b) Explain clearly Dijkstra's **single-source shortest path** algorithm for a graph with non-negative edge weights. Explain how the **priority queue** data structure is used for implementing this algorithm. Explain the complexity of this algorithm.

(5 marks)

—————**-END OF PAPER-**—————